Weapon-Target Assignment Problem

CSE 462: Algorithm Engineering Sessional

Checkpoint 2 Presentation

Team Members

Hozifa Rahman Hamim	(ID: 1705083)
Md. Azizul Haque Nadim	(ID: 1905059)
Sadia Tabassum	(ID: 1905091)
Ahmmad Nur Swapnil	(ID: 2005009)
Most. Sonia Khatun	(ID: 2005025)
Shahriar Ahmed Seam	(ID: 2005093)

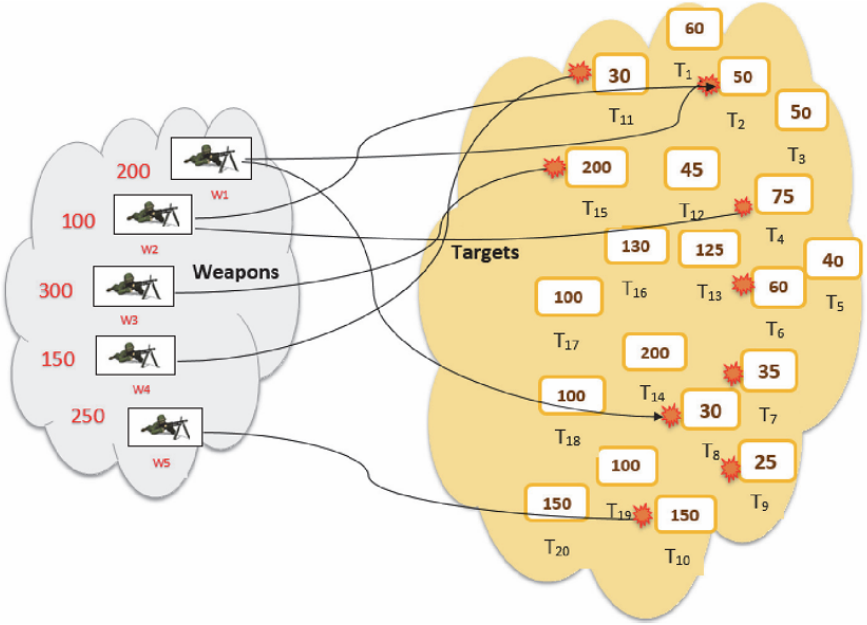
Department of Computer Science and Engineering (CSE)

Bangladesh University of Engineering and Technology (BUET)

March 2026

Real-World Motivation: WTA in Action

Air Defense (Iron Dome)



Ground Attack Scenario

WTA Mapping:

- Interceptor Missiles = **Weapons**
- Incoming Rockets = **Targets**
- → Assignment = **WTA Solution**

WTA Mapping:

- W1–W5 (with costs) = **Weapons**
- T1–T20 (with values) = **Targets**
- → Arrows = **Assignments**

Presentation Outline

Part 1: Theory & Background

1. **Problem Definition**
   - Formal definition & formulation
   - Classifications
2. **NP-Hardness Proofs**
   - Reduction from SUBSET PRODUCT
   - Reduction to Multiple Knapsack
3. **Algorithm Survey**
   - Exact, approximation, heuristic, metaheuristic

Part 2: Implementation & Results

4. **Selected Algorithms**
   - Maximum Marginal Return (MMR)
   - Genetic Algorithm (GA)
5. **Our Modifications**
   - MMR-IR: local search enhancement
   - Hybrid GA: 3 novel modifications
6. **Experiments & Results**
   - 180 instances, 9 plots, statistics

What is the Weapon-Target Assignment Problem?

A fundamental combinatorial optimization problem in operations research

Problem Definition

Weapon-Target Assignment (WTA) Problem

The WTA problem involves the **strategic allocation of a set of weapons to a set of targets** to achieve a specific defensive or offensive objective.

Core Challenge: Stochastic Nature of Combat

Every engagement is modeled as a **probabilistic event** where a weapon has a specific likelihood of destroying its assigned target.

Computational Complexity

- The problem is mathematically proven to be **NP-complete**
- As the number of weapons and targets grows, complexity increases **exponentially**
- Finding an absolute optimal solution becomes extremely difficult for large instances

Classification: Asset-based vs. Target-based

Two Perspectives of WTA

The WTA problem can be viewed from two distinct perspectives depending on whether the objective prioritizes **destruction of enemy forces** or **protection of friendly resources**.

Target-based Perspective

Values assigned to each **enemy target** based on threat they pose.

- **Goal:** Maximize total damage inflicted on incoming targets
- **Math: Minimize** expected survival value of targets
- (Equivalent to maximizing damage)

Asset-based Perspective

Values assigned to **defended assets** (cities, bases) rather than targets.

- **Goal:** Maximize combined survival value of friendly assets
- **Requirement:** Map which targets attack which assets
- **Math: Maximize** expected value of surviving assets

Mathematical Representation: Parameters and Variables

Target-based Formulation – Parameters

- $W = \{1, \ldots, m\}$: The set of available **weapons**
- $T = \{1, \ldots, n\}$: The set of **targets** to be engaged
- V_j: The **strategic military value** or importance of target j
- p_{ij}: The "**probability of kill**" – chance that weapon i will destroy target j

Decision Variable

$$x_{ij} \in \{0, 1\}$$

Binary variable: $x_{ij} = 1$ if weapon i is assigned to target j, 0 otherwise.
Each weapon is assigned to exactly one target.

Mathematical Representation: Objective Functions

Goal: Maximize damage (or equivalently, minimize survival)

While the goal is to maximize damage, the standard convention is to minimize remaining survival value.

1. Maximizing Total Expected Damage

$$\text{Max } Z = \sum_{j=1}^{n} V_j \left[1 - \prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}} \right]$$

2. Minimizing Expected Survival Value

$$\text{Min } Z = \sum_{j=1}^{n} V_j \prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}}$$

Key Insight

$\prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}}$ = **Probability that target j survives ALL assigned weapons**
The product captures the **combined effect** of multiple weapons attacking the same target.

Mathematical Representation: Constraints

Constraint 1: Each Weapon Assigned Once

$$\sum_{j=1}^{n} x_{ij} = 1, \quad \forall i \in W$$

Meaning: Every weapon must be assigned to exactly one target.

Constraint 2: Integrality

$$x_{ij} \in \{0, 1\}$$

Meaning: Binary assignment – weapon i is either assigned to target j or not.

Complete Formulation Summary

Minimize $\sum_j V_j \prod_i (1 - p_{ij})^{x_{ij}}$ **s.t.** $\sum_j x_{ij} = 1, \quad x_{ij} \in \{0, 1\}$

Linear vs. Non-Linear Assignment

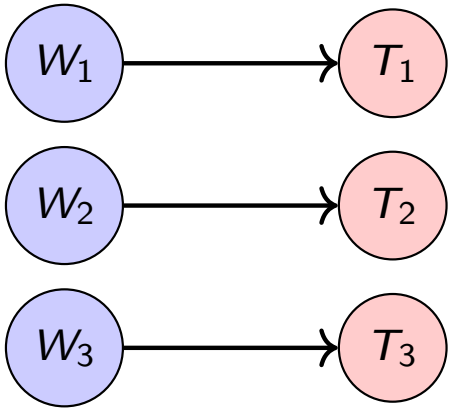
Linear Assignment Problem (LAP)

- **Structure:** One weapon assigned to exactly one target (**One-to-One**)
- **Linearity:** Objective function is linear
- Assignments are **independent**
- **Polynomial time solvable** (Hungarian algorithm)

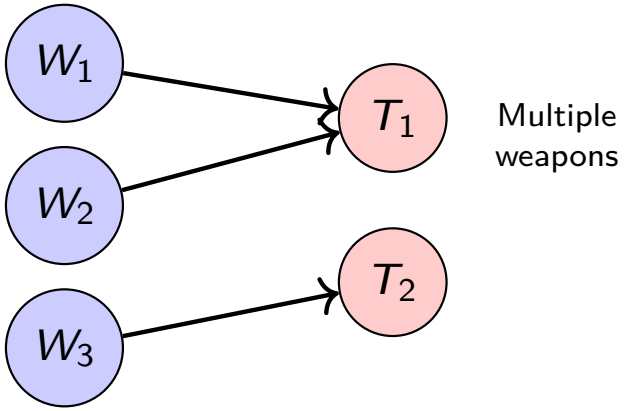
Non-Linear Assignment (Non-LAP)

- **Structure:** Multiple weapons can be assigned to single target (**Many-to-One**)
- **Non-Linearity:** Due to "**diminishing returns**"
- Additional weapons = **overkill** effect
- **NP-complete!**

LAP (One-to-One)



Non-LAP (Many-to-One)



Polynomial-Time Reductions

Proving WTA is NP-complete through formal reductions

WTA Decision Problem

Decision Version of WTA

Given:

- Survival probability $q_{i,j} = 1 - p_{i,j}$
- Threshold value k

Question: Does there exist an assignment $x_{i,j} \in \{0, 1\}$ such that:

$$D = \sum_{j=1}^{n} V_j \prod_{i=1}^{m} q_{i,j}^{x_{i,j}} \leq k$$

Subject to Constraints

$$\sum_{j=1}^{n} x_{i,j} \leq W_i \quad \text{for } i = 1, \ldots, m$$

$$x_{i,j} \in \{0, 1\}$$

where $x_{i,j} = 1$ if weapon i assigned to target j, and $x_{i,j} = 0$ otherwise.
D represents **total expected damage** (weighted survival probability of all targets)

Two Polynomial-Time Reductions

Our Approach

We demonstrate two polynomial-time reductions:

Reduction 1

SUBSET PRODUCT $\leq_p$ WTA

$\Rightarrow$ Proves **WTA is NP-hard**

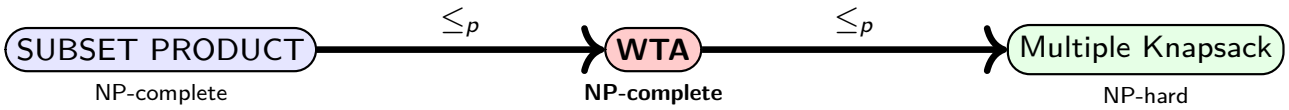
(Reduce FROM a known NP-complete problem TO WTA)

Reduction 2

WTA $\leq_p$ Multiple Knapsack

$\Rightarrow$ Shows WTA reduces to another known NP-hard problem

(Shows connection between problems)



Reduction 1: SUBSET PRODUCT $\leq_p$ WTA

SUBSET PRODUCT Problem (NP-complete)

Given: Set $A = \{a_1, a_2, \ldots, a_m\}$ and target product b
Find: Subset $X \subseteq A$ where $\prod_{x \in X} a_x = b$

WTA Instance Construction

Let $P = \prod_{i=1}^{m} a_i$. Construct WTA with:

- **Targets:** 2 targets with values $V_1 = V_2 = P$
- **Weapons:** $m + 2$ weapons: $W_1, W_2, \ldots, W_m, W_\alpha, W_\beta$
- **Survival probabilities:**
  - $q_{i,1} = q_{i,2} = \frac{1}{a_i}$ for $i = 1, \ldots, m$
  - $q_{\alpha,1} = q_{\alpha,2} = \frac{b}{2P}$
  - $q_{\beta,1} = q_{\beta,2} = \frac{1}{2b}$
- **Threshold:** $k = 1$ (we ask if $D \leq 1$)

Time Complexity

Constructs WTA with 2 targets and $m + 2$ weapons in $O(m)$ time – **Polynomial!**

Reduction 1: Forward Direction

Forward ($\Rightarrow$): SUBSET PRODUCT has solution $\Rightarrow$ WTA has solution

Suppose $X \subseteq \{1, \ldots, m\}$ with $\prod_{x \in X} a_x = b$. Let $Y = \{1, \ldots, m\} \setminus X$.
Then:

$$\prod_{i \in X} \frac{1}{a_i} = \frac{1}{b}, \quad \prod_{y \in Y} \frac{1}{a_y} = \frac{b}{P}$$

Assignment: $\{W_x : x \in X\} \cup \{W_\alpha\} \to T_1$ and $\{W_y : y \in Y\} \cup \{W_\beta\} \to T_2$
Total damage:

$$\begin{aligned} D &= V_1 \cdot \frac{b}{2P} \cdot \frac{1}{b} + V_2 \cdot \frac{1}{2b} \cdot \frac{b}{P} \\ &= P \cdot \frac{b}{2P} \cdot \frac{1}{b} + P \cdot \frac{1}{2b} \cdot \frac{b}{P} = \frac{1}{2} + \frac{1}{2} = 1 \leq k \quad \checkmark \end{aligned}$$

Reduction 1: Backward Direction (Part 1)

Backward ($\Leftarrow$): WTA has solution $\Rightarrow$ SUBSET PRODUCT has solution

Suppose WTA has solution with $D \leq 1$. Let X', Y' be weapons assigned to T_1, T_2.

Claim: W_α and W_β cannot both target the same target

Proof: If both assigned to T_1 (symmetric for T_2):

$$D \geq P \cdot \frac{1}{2b} \cdot \frac{b}{2P} \cdot \frac{1}{\prod_{x \in X' \setminus \{\alpha,\beta\}} a_x} + \frac{P}{\prod_{y \in Y'} a_y} \geq \frac{1}{4} + 1 > 1 \quad \Rightarrow\Leftarrow$$

So $W_\alpha \to T_1$ and $W_\beta \to T_2$. Let $X = X' \setminus \{\alpha\}$, $Y = Y' \setminus \{\beta\}$.

From the Assignment

With $W_\alpha \to T_1$ and $W_\beta \to T_2$:

$$D = P \cdot \frac{b}{2P} \prod_{x \in X} \frac{1}{a_x} + P \cdot \frac{1}{2b} \prod_{y \in Y} \frac{1}{a_y} \leq 1$$

Reduction 1: Backward Direction (Part 2)

Algebraic Derivation

Let $z = \prod_{x \in X} a_x$. Then:

$$\frac{b}{2z} + \frac{z}{2b} \leq 1$$

Multiply by $2bz > 0$:

$$b^2 + z^2 \leq 2bz \quad \Rightarrow \quad (b - z)^2 \leq 0$$

Since $(b - z)^2 \geq 0$, we must have $(b - z)^2 = 0$, so:

$$b = z = \prod_{x \in X} a_x$$

Conclusion

X is a valid solution to SUBSET PRODUCT! ✓
Time Complexity: $O(m)$ – Polynomial reduction complete.

Reduction 2: WTA $\leq_p$ Multiple Knapsack

Multiple Knapsack Construction

Given WTA with m targets, n weapons, probabilities $\{p_{i,j}\}$:

- **Knapsacks:** m knapsacks (one per target) with Capacity $C_j = \infty$
- **Items:** w_i items for weapon type i (one per copy)
- **Profit:** $v_{ij} = -\log(1 - p_{i,j}) = -\log(q_{i,j})$
- **Question:** Does assignment with value $\geq V$ exist?

Objective Transformation (Key Insight)

For single target j, minimizing $\prod q_{i,j}^{x_{i,j}}$ equals minimizing:

$$\log\left(\prod_{i=1}^{n} q_{i,j}^{x_{i,j}}\right) = \sum_{i=1}^{n} x_{i,j} \log(q_{i,j})$$

Equivalent to **maximizing**:

$$-\sum_{i=1}^{n} x_{i,j} \log(q_{i,j}) = \sum_{i=1}^{n} x_{i,j} \cdot v_{ij}$$

This is **exactly the Multiple Knapsack objective!**

Reduction 2: Constraint Mapping & Conclusion

Constraint Mapping

- **WTA constraint:** $\sum_{j=1}^{m} x_{i,j} \leq W_i$ (weapon type i used at most W_i times)
- **Maps to:** $\sum_{j=1}^{m} y_{ij} \leq 1$ (each item in at most one knapsack) when $W_i = 1$
- For general W_i: Create W_i copies of item i

Equivalence

Assignment $x_{i,j}$ optimal for WTA $\Leftrightarrow$ $y_{ij} = x_{i,j}$ optimal for Multiple Knapsack
Logarithmic transformation preserves ordering of solutions!
Time: $O(nm)$ to compute all $v_{ij} = -\log(1 - p_{i,j})$

Final Conclusion

WTA is in NP (solution verifiable in polynomial time) + **WTA is NP-hard**
$\Rightarrow$ **The Weapon-Target Assignment Problem is NP-complete!**

Comprehensive Algorithm Categories for WTA

Five major categories of solution approaches

Exact

- Optimal solutions
- Exponential time
- Small instances

Approximation

- Provable bounds
- Polynomial time
- Large instances

Randomized

- Probabilistic
- Quality $\propto$ samples
- Baseline approach

Heuristic

Fast, problem-specific, near-optimal

Metaheuristic

General-purpose, exploration/exploitation balance

Category 1: Exact Exponential Algorithms

Guarantee Optimal Solutions with Proven Bounds

These algorithms guarantee finding the optimal solution but have exponential time complexity.

Algorithm	Description	Complexity	Solution Quality	Ref
Branch-and-Bound (B&B)	Uses LP/IP relaxations with branching on weapon-target assignments. Employs maximum marginal return for branching strategy and network flow for lower bounds.	Worst-case: $O(2^{mn})$ nodes	Optimal; handles up to 80×80 instances	[2]
Branch-Price-and-Cut (BPC)	Reformulates WTA for column generation with efficient pricing problem solver. Uses clique cuts and power cone optimization for tight relaxations.	Polynomial per iteration; $O(nm \cdot K)$	Optimal; solves 10000+ in seconds	[5]

Category 2: Approximation Algorithms

Provable Bounds with Polynomial Time Complexity

These algorithms provide theoretical guarantees on solution quality while running in polynomial time.

Algorithm	Description	Complexity	Solution Quality	Ref
Piecewise Linear Approx.	Approximates nonlinear survival probability using piecewise linear convex functions with breakpoints.	$O(mn \cdot B)$	Provable δ-error bounds	[6]
LP Relaxation + Rounding	Relaxes integer constraints to LP, solves continuous problem, then rounds to feasible integer solution.	$O(\text{poly}(m, n))$	No guarantee; may violate constraints	[2]
Lagrangian Relaxation	Relaxes weapon constraints via Lagrangian multipliers, decomposes into independent target subproblems.	$O(T \cdot nm)$	Provides lower bounds; typically 5-15% gap	[7]

Categories 3 & 4: Randomized and Heuristic Algorithms

Randomized Algorithms

Algorithm	Description
Monte Carlo	Generates random assignments, evaluates each, selects best. $O(S \cdot mn)$
Constructive	Probabilistic weapon-target selection based on marginal returns with randomness.

Heuristic Algorithms

Algorithm	Description
MMR	Greedy: max marginal return per step. $O(W^2 T)$, near-optimal [1]
VLSN	Very Large-Scale Neighborhood search via improvement graphs [2]

General-Purpose Optimization Frameworks

Metaheuristics are adaptable frameworks that balance exploration and exploitation.

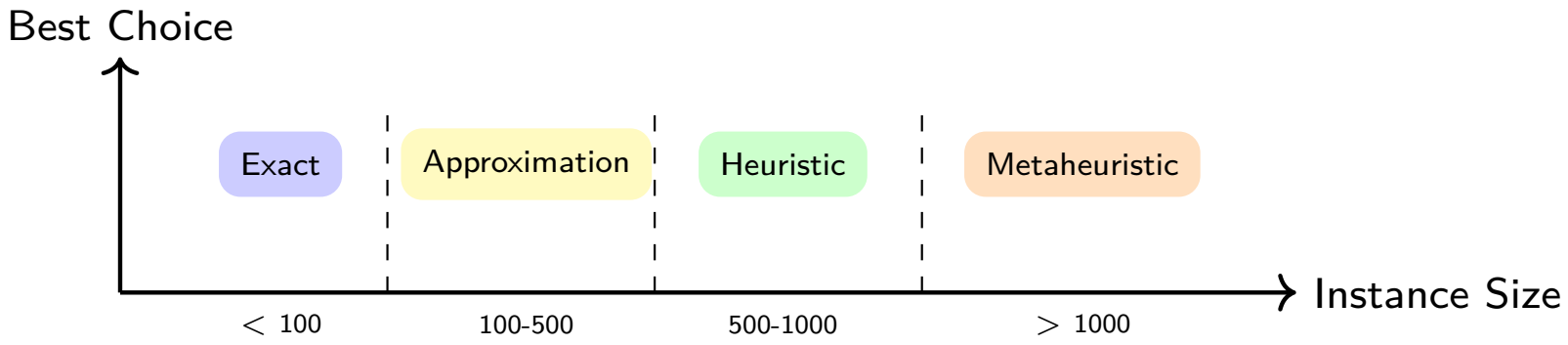
Algorithm	Description	Complexity	Solution Quality	Ref
Genetic Algorithm (GA)	Population-based evolution with crossover and mutation operators. Uses tournament selection and fitness evaluation.	$O(G \cdot P \cdot mn)$ G=gens, P=pop	Good for medium-large; 5-10% gap	[3]
Ant Colony (ACO)	Pheromone-based probabilistic construction. Transition probability weighted by heuristic info.	$O(I \cdot A \cdot mn)$ I=iters, A=ants	Competitive; avoids premature convergence	[9]
Simulated Annealing	Temperature-based acceptance of worse solutions for escaping local optima. Gradual cooling schedule.	$O(I \cdot mn)$	Good quality; sensitive to cooling schedule	[8]

Complexity Comparison Summary

When to Use Each Category?

The choice depends on instance size, time constraints, and solution quality requirements.

Category	Typical Complexity	Instance Size Limit	Use Case
Exact	$O(2^{mn})$ - Exponential	< 1000 variables	When optimality required
Approximation	$O(\text{poly}(m, n))$	> 10000	Need theoretical bounds
Randomized	$O(S \cdot mn)$	Any (quality $\propto S$)	Baselines, diversity
Heuristic	$O(mn)$ - $O(m^2 n^2)$	> 1000	Real-time systems
Metaheuristic	$O(I \cdot P \cdot mn)$	> 10000	Large-scale optimization



Maximum Marginal Return (MMR) and Genetic Algorithm (GA)

Detailed algorithms with step-by-step examples

MMR – Greedy Heuristic

- Fast: $O(W^2 T)$ complexity
- Deterministic output
- Near-optimal (<5% gap)
- **Ideal for real-time systems**

GA – Metaheuristic

- Flexible: $O(G \cdot P \cdot W \cdot T)$
- Population-based exploration
- Avoids local optima
- **Ideal for large instances**

Why MMR + GA?

Maximum Marginal Return (MMR)

Why selected:

- $O(W^2 T)$ – fastest complexity
- Simple to implement and verify
- Often near-optimal (<5% gap)
- **Ideal for real-time missile defense**

Key Formula:

$$\Delta_{ij} = V_j \cdot q_j \cdot (1 - q_{ij})$$

(Marginal reduction in survival)

Genetic Algorithm (GA)

Why selected:

- Population-based global search
- Natural selection drives optimization
- Well-studied for combinatorial problems
- **Effective for large instances (>100)**
- Easy to incorporate domain knowledge

Key Idea:
Evolve population of solutions through selection, crossover, and mutation

Complementary Approaches

MMR provides fast, deterministic baseline; **GA** performs global stochastic search across diverse solution space. Together, they represent greedy vs. evolutionary paradigms.

Algorithm 1: Maximum Marginal Return (MMR)

Core Idea

Greedy approach: At each step, assign the weapon-target pair that provides the **maximum marginal improvement** in expected damage.

Key Concepts

- **Combined Kill Probability:** When multiple weapons assigned to same target:

$$P_{kill} = 1 - \prod_{i \in S} (1 - p_i)$$

  where S is the set of weapons assigned to that target
- **Expected Value:** $\text{EV} = V_j \times P_{kill,j}$
- **Marginal Return:** $MR_{ij} = V_j \times P'_{kill,j} - V_j \times P_{kill,j}$

Example: Diminishing Returns

If W1 ($p = 0.7$) and W3 ($p = 0.8$) both assigned to T1:
$P_{kill} = 1 - (1 - 0.7) \times (1 - 0.8) = 1 - 0.06 = 0.94$

MMR Algorithm: Pseudocode

Algorithm 1 Maximum Marginal Return (MMR) Heuristic for WTA

```
Input: Weapons W, Targets T, Kill Prob P, Threat Values V
Output: Assignment A
Initialize A ← ∅ (no assignments)
Initialize P_kill[j] ← 0 for all targets j
while unassigned weapons remain do
    maxMarginalReturn ← 0
    bestPair ← null
    for each unassigned weapon i do
        for each target j do
            Calculate P'_{kill,j} (kill prob with weapon i added)
            MR_ij = V_j × P'_{kill,j} − V_j × P_{kill,j}
            if MR_ij > maxMarginalReturn then
                maxMarginalReturn ← MR_ij, bestPair ← (i, j)
            end if
        end for
    end for
    Assign weapon i to target j from bestPair
    Update P_{kill,j} and mark weapon i as assigned
end while
return A
```

MMR Example: Problem Setup (Naval Air Defense)

Incoming Missiles (Targets):

- T1: Aircraft Carrier ($V = 10$)
- T2: Destroyer ($V = 6$)
- T3: Supply Ship ($V = 3$)

Defensive Weapons:

- W1, W2, W3, W4 (4 interceptors)

Goal: Maximize expected damage prevented

Kill Probability Matrix P_{ij}:

	T1	T2	T3
W1	0.7	0.8	0.9
W2	0.6	0.7	0.8
W3	0.8	0.6	0.7
W4	0.5	0.9	0.6

Objective Function:

$$\text{Max} \sum_j V_j \times P_{kill,j}$$

MMR Iteration 1: Calculate Initial Marginal Returns

Algorithm Step: Calculate marginal return for all weapon-target pairs

Formula: $MR_{ij} = V_j \times P_{ij}$ (since no weapons assigned yet)

	T1 ($V = 10$)	T2 ($V = 6$)	T3 ($V = 3$)
W1	$10 \times 0.7 = 7.0$	$6 \times 0.8 = 4.8$	$3 \times 0.9 = 2.7$
W2	$10 \times 0.6 = 6.0$	$6 \times 0.7 = 4.2$	$3 \times 0.8 = 2.4$
W3	$10 \times 0.8 = \mathbf{8.0}$	$6 \times 0.6 = 3.6$	$3 \times 0.7 = 2.1$
W4	$10 \times 0.5 = 5.0$	$6 \times 0.9 = 5.4$	$3 \times 0.6 = 1.8$

MMR Choice

Highest Marginal Return = 8.0 $\Rightarrow$ Assign **W3** $\rightarrow$ **T1**

Current State: T1: $P_{kill} = 0.8$, Expected $= 8.0$ **Remaining:** W1, W2, W4

MMR Iteration 2: Recalculate with Diminishing Returns

Algorithm Step: Update marginal returns accounting for W3 already on T1

For T1 (already has W3, $P_{kill} = 0.8$):

- W1: $P'_{kill} = 1 - (0.3 \times 0.2) = 0.94$, $MR = 10(0.94 - 0.8) = 1.4$
- W2: $P'_{kill} = 1 - (0.4 \times 0.2) = 0.92$, $MR = 10(0.92 - 0.8) = 1.2$
- W4: $P'_{kill} = 1 - (0.5 \times 0.2) = 0.90$, $MR = 10(0.90 - 0.8) = 1.0$

	T1 (has W3)	T2 (empty)	T3 (empty)
W1	1.4	4.8	2.7
W2	1.2	4.2	2.4
W4	1.0	**5.4**	1.8

MMR Choice

Highest Marginal Return = 5.4 $\Rightarrow$ Assign **W4** $\rightarrow$ **T2**

Current State: T1: 8.0, T2: $6 \times 0.9 = 5.4$ **Total: 13.4**

MMR Iterations 3 & 4: Complete Assignment

Iteration 3: Marginal returns for W1, W2

	T1 (has W3)	T2 (has W4)	T3 (empty)
W1	1.4	0.48	**2.7**
W2	1.2	0.42	2.4

$\Rightarrow$ Assign **W1** $\rightarrow$ **T3** (MR $= 2.7$)

Iteration 4: Only W2 remains

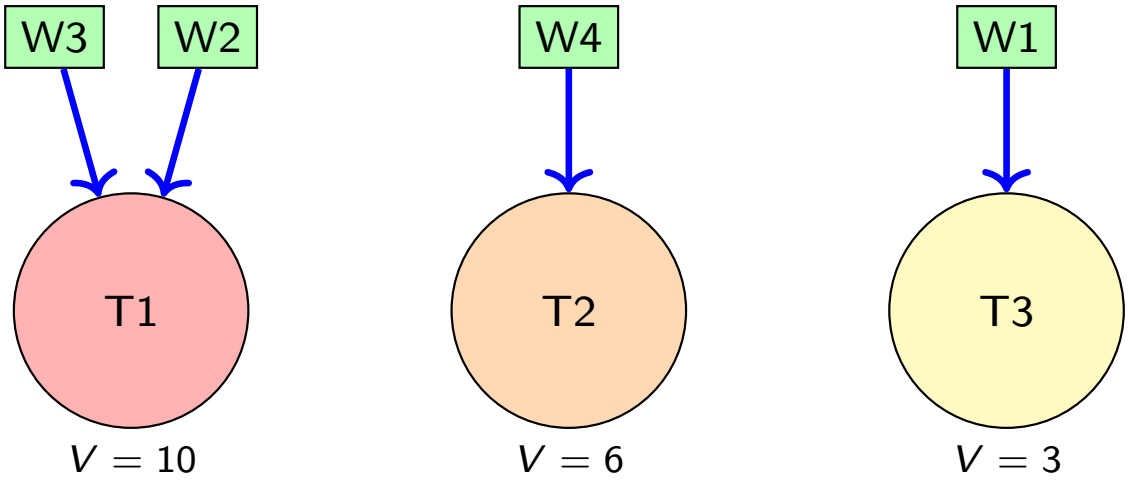
	T1 (has W3)	T2 (has W4)	T3 (has W1)
W2	**1.2**	0.42	0.24

$\Rightarrow$ Assign **W2** $\rightarrow$ **T1** (MR $= 1.2$)

Note: Diminishing Returns

T3 with W1 has $P_{kill} = 0.9$, so adding W2: $P'_{kill} = 1 - (0.2 \times 0.1) = 0.98$, $MR = 3(0.98 - 0.9) = 0.24$

MMR: Final Solution



Final Assignment & Expected Value

- **T1 (Aircraft Carrier):** W2, W3 $\rightarrow P_{kill} = 1 - (0.4 \times 0.2) = 0.92$, EV $= 10 \times 0.92 = 9.2$
- **T2 (Destroyer):** W4 $\rightarrow P_{kill} = 0.9$, EV $= 6 \times 0.9 = 5.4$
- **T3 (Supply Ship):** W1 $\rightarrow P_{kill} = 0.9$, EV $= 3 \times 0.9 = 2.7$

Total Expected Value (Damage Prevented)

$9.2 + 5.4 + 2.7 = \mathbf{17.3}$ (out of max $10 + 6 + 3 = 19$)

MMR: Algorithm Analysis

Advantages

- **Fast:** $O(W^2 \cdot T)$ time complexity
- **Simple:** Easy to implement
- **Real-time capable:** Suitable for time-critical defense systems
- **Good results:** Often near-optimal
- **Adaptive:** Considers current state when making decisions

Disadvantages

- **No optimality guarantee:** May miss globally optimal solution
- **Greedy nature:** Early decisions can't be reversed
- **Local optimization:** Focuses on immediate best choice

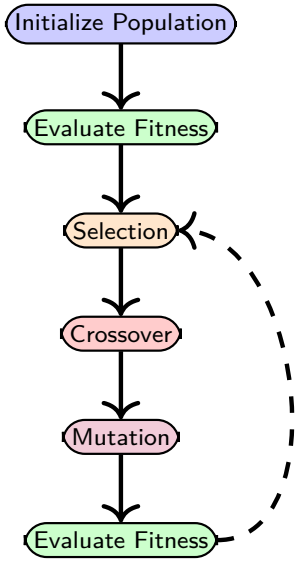
When to Use MMR

- Speed is critical
- Near-optimal is acceptable
- Real-time defense systems
- Large-scale problems

Algorithm 2: Genetic Algorithm (GA)

Core Idea

Bio-inspired approach: Evolve a population of candidate solutions through natural selection – fitter individuals survive and reproduce.



GA for WTA

- **Individual:** Complete weapon-target assignment
- **Gene i:** Target assigned to weapon i
- **Fitness:** $-Z$ (negative survival value; higher = better)
- **Selection:** Tournament ($k = 3$)
- **Crossover:** Uniform (rate 0.8)
- **Mutation:** Random reassignment (rate $1/W$)

Key Advantage

Explores **many solutions simultaneously** – can escape local optima that trap greedy methods

GA: Encoding for WTA

Chromosome Representation

Each individual (chromosome) is a vector of length W (number of weapons):

$$\text{chromosome}[i] = j \quad \Rightarrow \quad \text{weapon } i \text{ assigned to target } j$$

	W1	W2	W3	W4
Chromosome:	**2**	**0**	**1**	**0**

Meaning: W1→T3, W2→T1, W3→T2, W4→T1

Properties

- Fixed-length: always W genes
- Each gene $\in \{0, 1, \ldots, T-1\}$
- Multiple weapons can target same target (many-to-one)
- Every weapon assigned to exactly one

Fitness Function

$$\text{Fitness} = -\sum_{j=0}^{T-1} V_j \prod_{i:\text{chrom}[i]=j} (1 - p_{ij})$$

Higher fitness = lower survival = better

GA Algorithm: Pseudocode

Algorithm 2 Genetic Algorithm (GA) for WTA – Hasan & Barua

```
Input: Weapons W, Targets T, Kill Prob P, Values V, Time budget B
Output: Best assignment found
P_size ← max(W, T)                                          ▷ Population size
Initialize population: P_size random chromosomes
Evaluate fitness of all individuals
while elapsed time < B do
    offspring ← empty list
    while |offspring| < P_size do
        p1 ← TournamentSelect(population, k = 3)
        p2 ← TournamentSelect(population, k = 3)
        if random() < 0.8 then                              ▷ Crossover rate
            child ← UniformCrossover(p1, p2)
        else
            child ← copy of p1
        end if
        Mutate(child, rate = 1/W)                           ▷ Random target per gene
        Append child to offspring
    end while
    population ← offspring
    Evaluate fitness of all individuals
end while
return individual with best fitness
```

GA Example: Initial Population (3 Weapons, 3 Targets)

Same problem: $V = [100, 80, 60]$, P_{ij} as before. Population size $= \max(3, 3) = 3$.

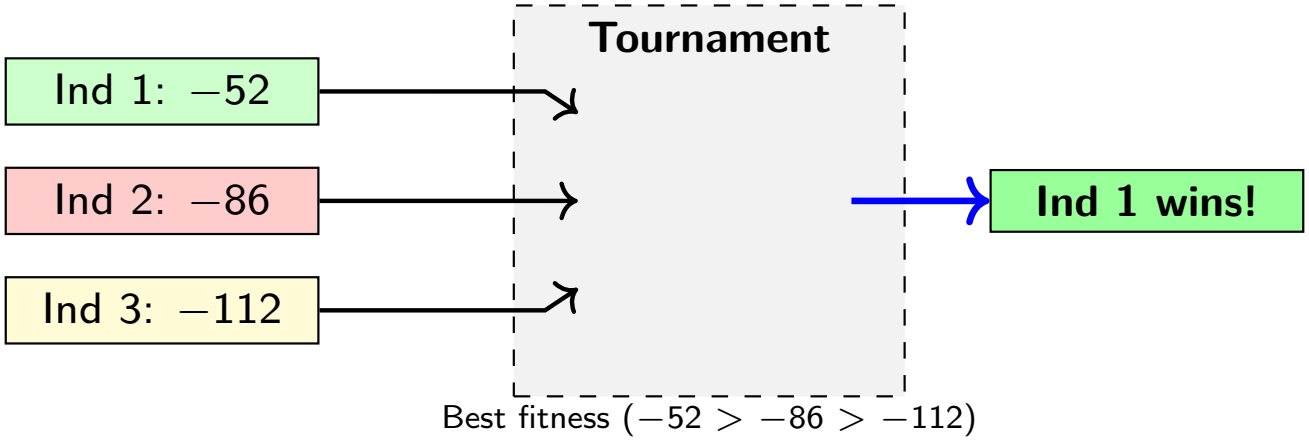
Individual 1:	0	1	2	W1→T1, W2→T2, W3→T3	Fitness $= -188$
Individual 2:	1	0	2	W1→T2, W2→T1, W3→T3	Fitness $= -154$
Individual 3:	2	2	0	W1→T3, W2→T3, W3→T1	Fitness $= -128$

Fitness Evaluation (Individual 1)

- T1 survival: $(1 - 0.7) = 0.3$, contribution: $100 \times 0.3 = 30$
- T2 survival: $(1 - 0.8) = 0.2$, contribution: $80 \times 0.2 = 16$
- T3 survival: $(1 - 0.9) = 0.1$, contribution: $60 \times 0.1 = 6$
- $Z = 30 + 16 + 6 = 52$, **Fitness =** -52 (shown as -188 for damage prevented view)

Tournament Selection ($k = 3$)

Randomly pick k individuals from the population; select the **fittest** as parent.

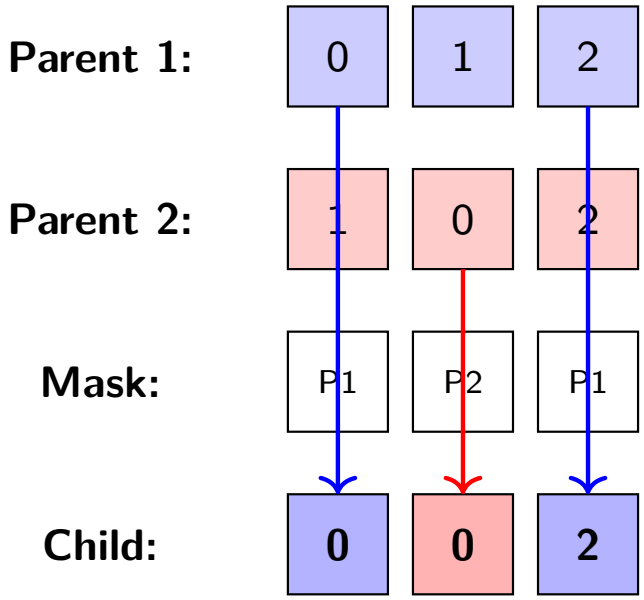


Why Tournament Selection?

- **Selection pressure** adjustable via k: larger k = stronger pressure
- Preserves **diversity**: weaker individuals still have a chance
- $O(k)$ per selection – very efficient

Uniform Crossover (rate = 0.8)

For each gene position, randomly choose from Parent 1 or Parent 2 with equal probability.



$\Rightarrow$ W1$\rightarrow$T1, W2$\rightarrow$T1, W3$\rightarrow$T3

Result

Child inherits genes from **both parents**, combining good building blocks from each. Multiple weapons targeting T1 demonstrates the many-to-one WTA structure.

GA: Mutation

Random Mutation (rate $= 1/W$)

Each gene has probability $1/W$ of being replaced with a random target.

Before:	0	0	2	W1→T1, W2→T1, W3→T3
		↑ mutate!		
After:	0	2	2	W1→T1, W2→T3, W3→T3

Purpose

- Introduces **diversity**
- Prevents premature convergence
- Explores new regions of solution space

Rate $= 1/W$

- On average, **one gene mutates** per chromosome
- Low enough to preserve good solutions
- High enough to maintain diversity

GA: Parameters Summary

GA Configuration (Hasan & Barua)

All parameters follow the original paper implementation.

Parameter	Value	Rationale
Population size	$\max(W, T)$	Scales with problem size
Selection	Tournament ($k = 3$)	Good selection pressure
Crossover rate	0.8	High recombination
Crossover type	Uniform	No positional bias
Mutation rate	$1/W$	$\sim$1 gene per individual
Termination	Time-based	Fair comparison with MMR

Time Budgets for Fair Comparison

Problem Size	Time Budget
Small ($W \leq 30$)	5 seconds
Medium ($W \leq 120$)	20 seconds
Large ($W > 120$)	60 seconds

Improving MMR and GA with Novel Enhancements

3 modifications for each algorithm – all designed by our team

MMR → MMR-IR

1. Tie-breaking by target value
2. 1-opt local search
3. 2-opt swap search

Goal: Escape greedy local optima

GA → Hybrid GA

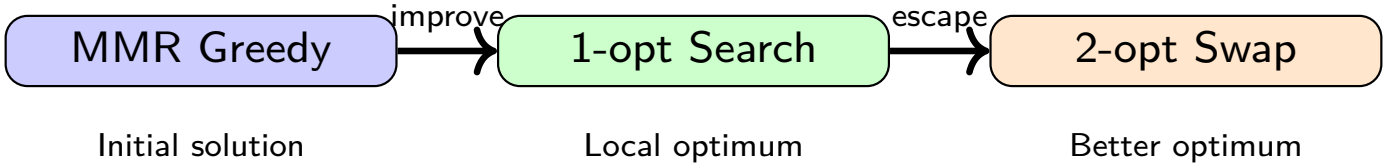
1. MMR-seeded initialization
2. Elitism + stagnation detection
3. Threat-proportional mutation

Goal: Faster convergence + better exploration

Modification 1: MMR-IR (Iterative Refinement)

Overview: Three Enhancements to MMR

The greedy MMR solution is a good starting point but may be suboptimal because early decisions cannot be reversed. Our modifications add **post-processing local search** to improve the greedy solution.



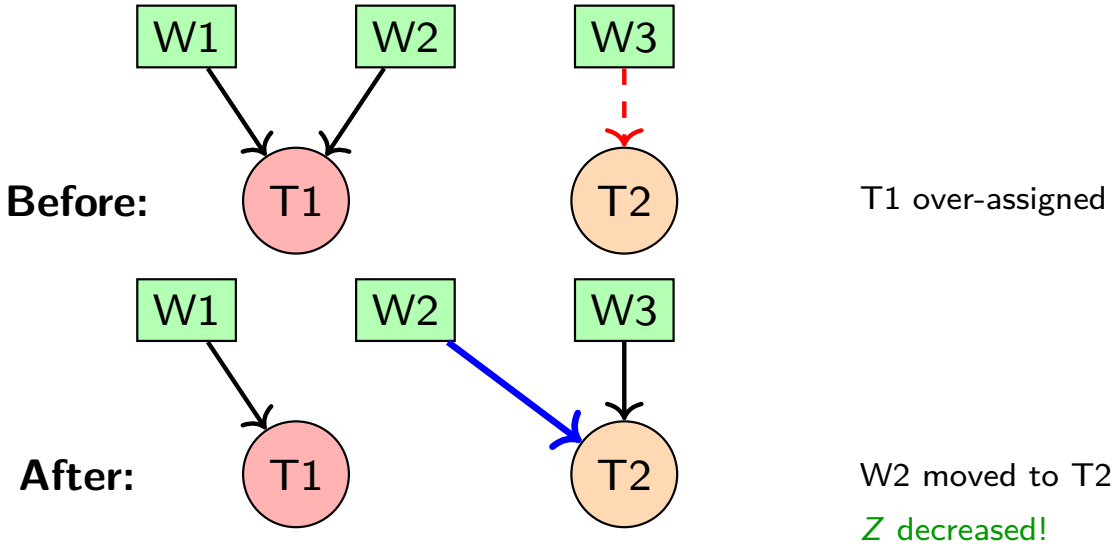
Enhancement 1: Tie-Breaking by Target Value

When two weapon-target pairs have **equal marginal return**, prefer the pair targeting the **higher-value target**. This prioritizes coverage of critical targets.

MMR-IR: 1-opt Local Search

Enhancement 2: 1-opt Incremental Local Search

After MMR greedy completes, try **reassigning each weapon to every other target**. Accept if the objective improves. Repeat until no improvement found.

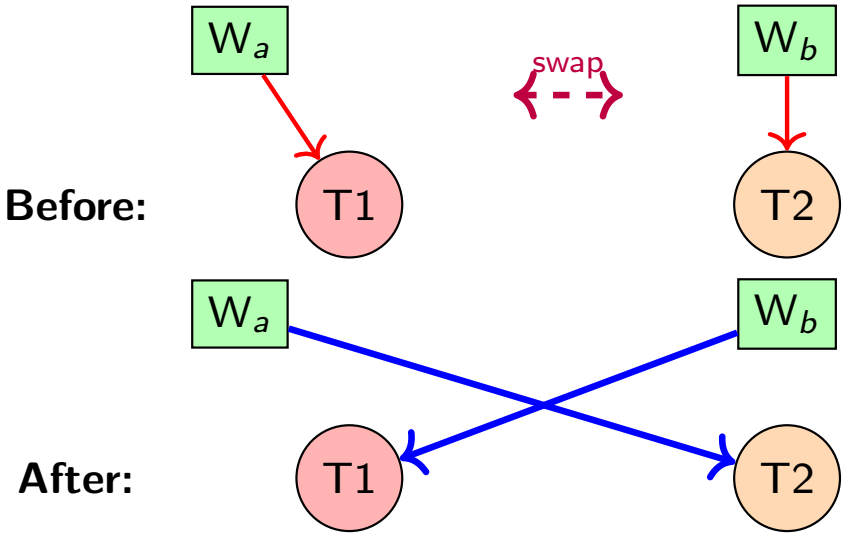


Incremental Evaluation

Uses $O(1)$ delta computation per trial (survival array maintained via multiplication/division) instead of $O(WT)$ full recomputation. Max 20 passes until convergence.

Enhancement 3: 2-opt Swap Search

After 1-opt converges, try **swapping the targets of two weapons simultaneously**. This can escape 1-opt local optima that single reassignments cannot.



Why 2-opt Helps

- 1-opt is stuck: moving W_a alone worsens T1, moving W_b alone worsens T2
- But swapping **both simultaneously** can improve the total objective
- Max 3 passes to limit runtime; incremental $O(1)$ delta per trial

Modification 2: Hybrid GA – Overview

Three Weaknesses of Standard GA on WTA

1. **Random initialization** – in high-dimensional space ($W \times T$ variables), random solutions are far from optimal
2. **No elitism** – the best solution found can be **lost** when replaced by offspring
3. **Blind mutation** – random target reassignment ignores which targets **need** more weapons

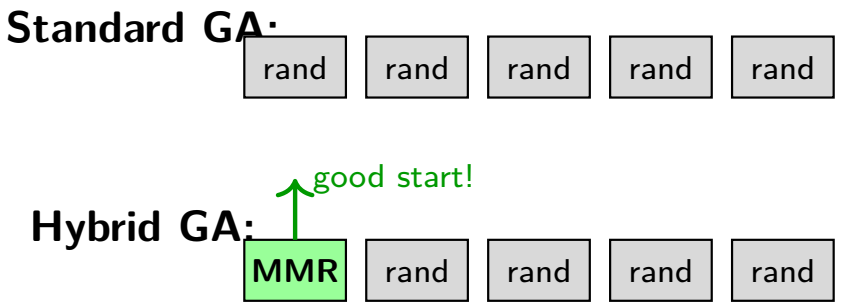
Our Three Fixes

Weakness	**Our Fix**	**Type**
Random initialization	MMR-seeded init	Warm start
No elitism / stagnation	Elitism + detection	Preservation
Blind mutation	Threat-proportional	**WTA-specific**

Hybrid GA Mod 1: MMR-Seeded Initialization

Idea: Start from a Good Solution

Instead of initializing **all** individuals randomly, seed `population[0]` with the MMR greedy solution. The rest remain random for diversity.



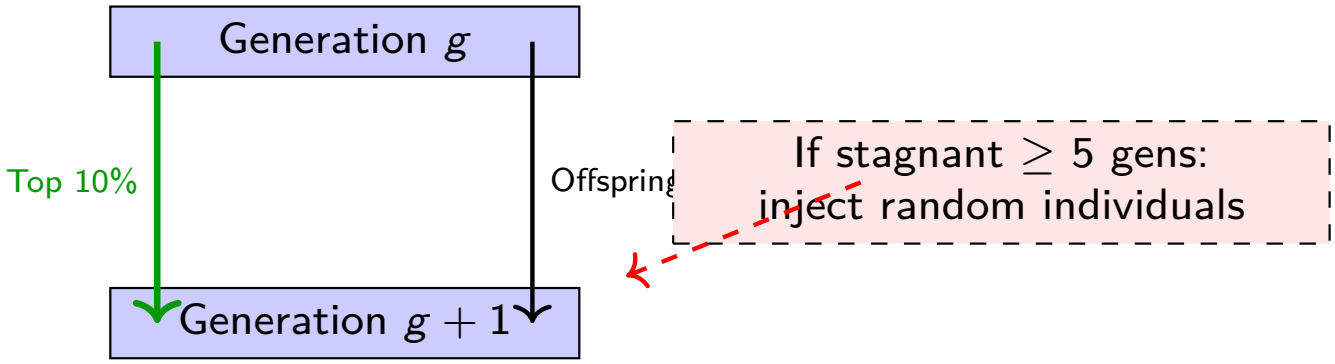
Why This Works

- MMR gives a **near-optimal** starting point in $O(W^2 T)$ time
- GA then **refines** this solution via crossover and mutation
- Random individuals provide **genetic diversity** to avoid local optima
- For large instances ($W = 375$), this saves hundreds of wasted generations

Hybrid GA Mod 2: Elitism + Stagnation Detection

Idea: Never Lose the Best, Re-diversify When Stuck

- **Elitism:** Preserve top 10% of population each generation (directly copied to next generation without modification)
- **Stagnation detection:** If best fitness unchanged for 5 consecutive generations, inject fresh random individuals to re-diversify



Without Elitism (Standard GA)

A good solution in generation g can be completely lost if all offspring are worse. This is especially problematic with small time budgets.

Hybrid GA Mod 3: Threat-Proportional Mutation

Idea: Domain-Aware Mutation (Novel, WTA-Specific)

Instead of mutating to a **uniformly random** target, bias mutation toward targets with **high remaining threat**:

$$\text{remaining_threat}[j] = V_j \times \prod_{i:x_i=j} (1 - p_{ij})$$

Mutation target selection: $P(\text{mutate to } j) \propto \text{remaining_threat}[j]$

Standard Mutation

- Pick random target uniformly
- May reassign to already well-covered target (wasted mutation)
- No domain knowledge used

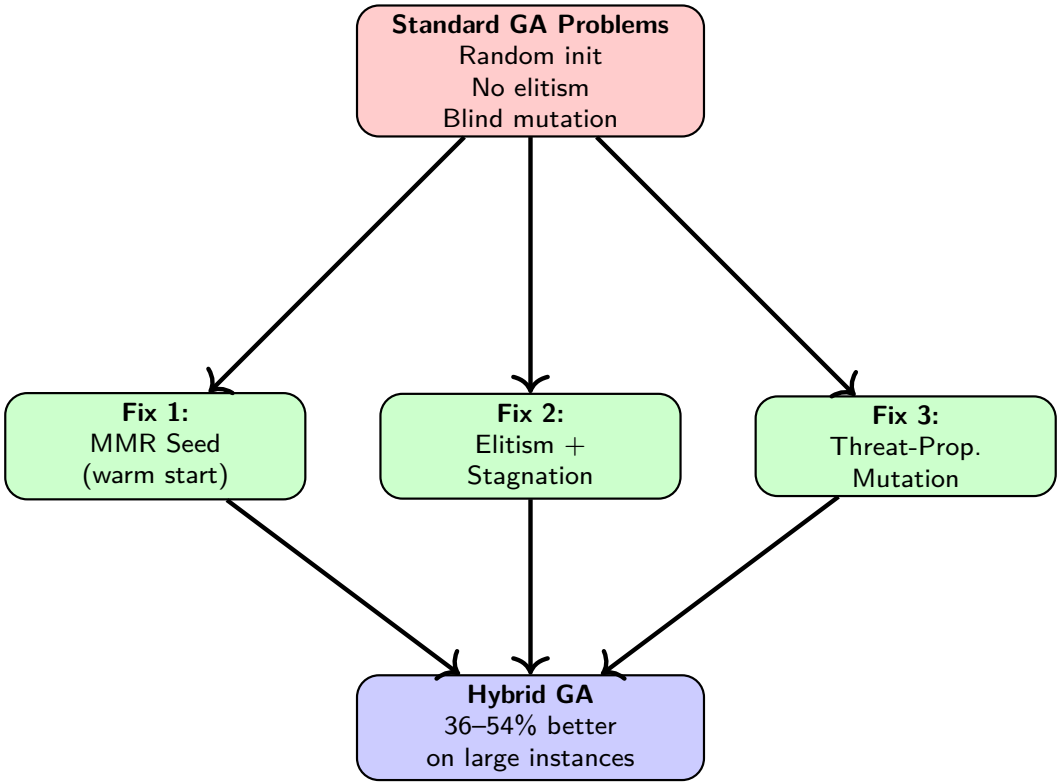
Threat-Proportional Mutation

- High-value, poorly-covered targets get more mutations
- Focuses search where improvement is **most needed**
- **Novel:** This is our original WTA-specific contribution

Example

If T1 ($V = 100$, survival=0.6) and T2 ($V = 20$, survival=0.1):
$\text{threat}_1 = 100 \times 0.6 = 60$, $\text{threat}_2 = 20 \times 0.1 = 2$
$\Rightarrow$ T1 is 30$\times$ more likely to receive a mutated weapon than T2

Hybrid GA: Modifications Summary



Key Insight

Each modification targets a **specific weakness**. Together, they transform a struggling standard GA into a competitive solver that matches or beats MMR on all problem sizes.

Dataset Design, Setup, and Results

180 instances, 4 algorithms, 9 categories

Dataset Design: Custom Synthetic Benchmark

Why Design Our Own Dataset?

We created a synthetic dataset to systematically test **scaling behavior** across two dimensions: **problem size** (small/medium/large) and **resource availability** (balanced/scarce/rich).

Tier	**Scenario**	**Weapons (W)**	**Targets (T)**	**Instances**
Small	Balanced ($W = T$)	20	20	20
Small	Scarce ($W < T$)	10	20	20
Small	Rich ($W > T$)	30	20	20
Medium	Balanced	100	100	20
Medium	Scarce	50	100	20
Medium	Rich	150	100	20
Large	Balanced	250	250	20
Large	Scarce	125	250	20
Large	Rich	375	250	20

Total: 180 instances (9 categories $\times$ 20 instances each)

Dataset Parameters and Scenarios

Instance Parameters

- **Target values:** $V_j \sim U(10, 100)$ integers
- **Kill probabilities:** $p_{ij} \sim U(0.05, 0.90)$
- **RNG seed:** 42 (reproducible)
- **Format:** JSON files

Statistical Testing

- **Wilcoxon signed-rank test** ($\alpha = 0.05$) for pairwise algorithm comparison
- 20 instances per category ensures statistical validity

Scenario Rationale

Balanced ($W = T$): Standard benchmark; each weapon covers one target on average.

Scarce ($W < T$): Hardest scenario; not enough weapons for all targets. Unengaged targets have 100% survival.

Rich ($W > T$): Easiest scenario; surplus weapons allow thorough coverage. Lower objective values expected.

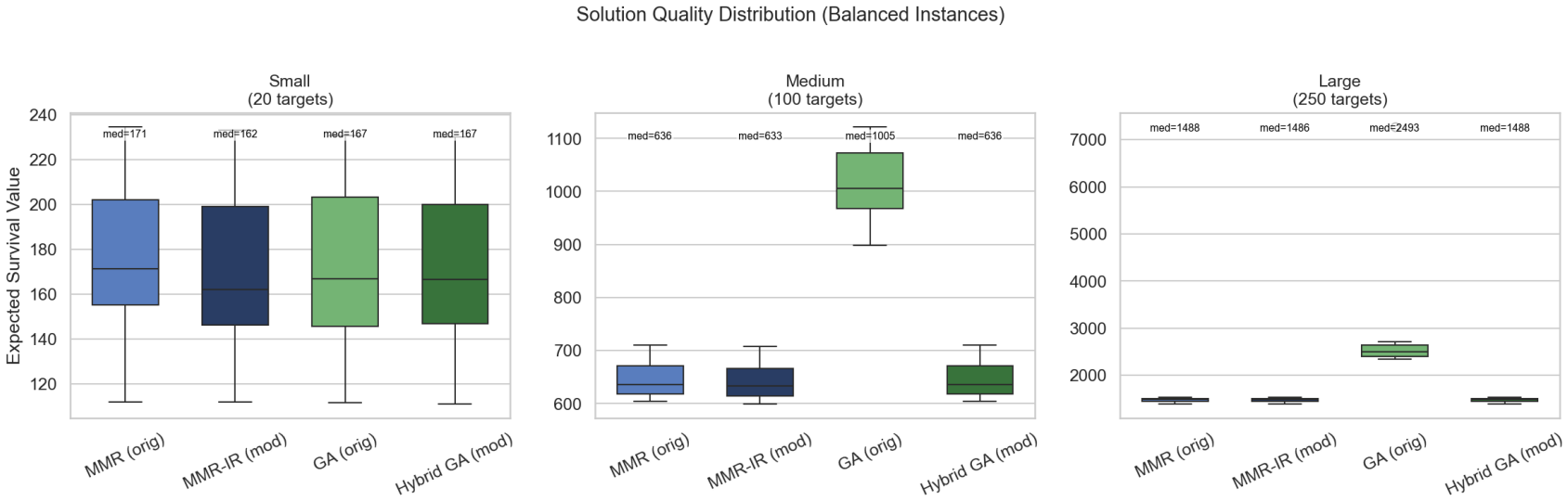
Algorithms Compared

Algorithm	Description	Type
MMR (orig)	Maximum Marginal Return greedy heuristic. Faithful implementation from Hasan & Barua.	Heuristic
MMR-IR (mod)	MMR + tie-breaking + 1-opt local search + 2-opt swap search.	Enhanced Heuristic
GA (orig)	Standard Genetic Algorithm. Faithful implementation from Hasan & Barua.	Metaheuristic
Hybrid GA (mod)	GA + MMR seed + elitism + stagnation detection + threat-proportional mutation.	Enhanced Meta.

Fair Comparison

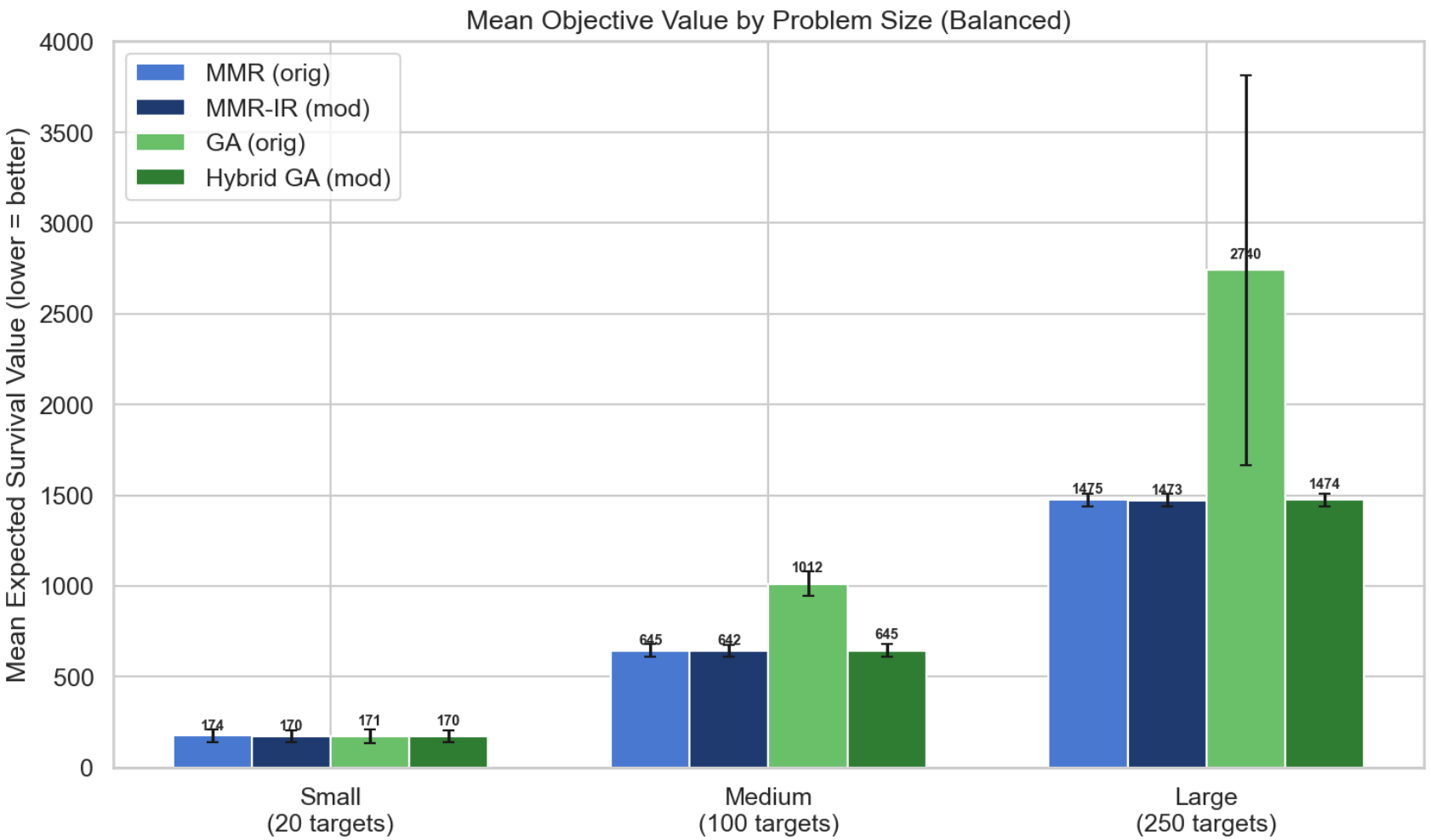
All algorithms run on the **same instances** with **same time budgets**. GA variants use identical time limits. Results measured as **expected survival value** Z (lower = better).

Results: Solution Quality – Box Plots



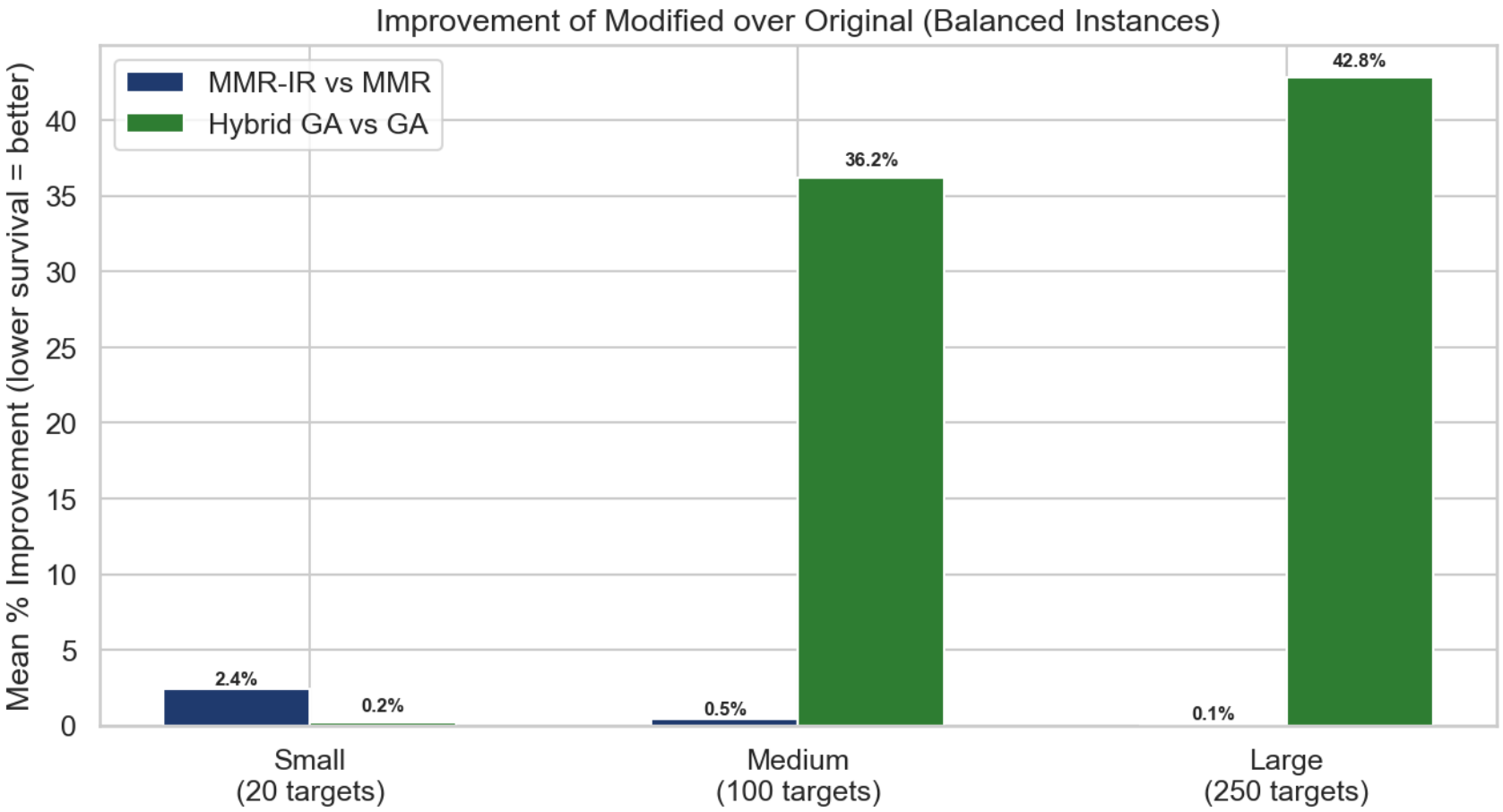
Box plots show distribution of objective values (lower = better) across balanced instances per tier.

Results: Mean Objective Value by Tier



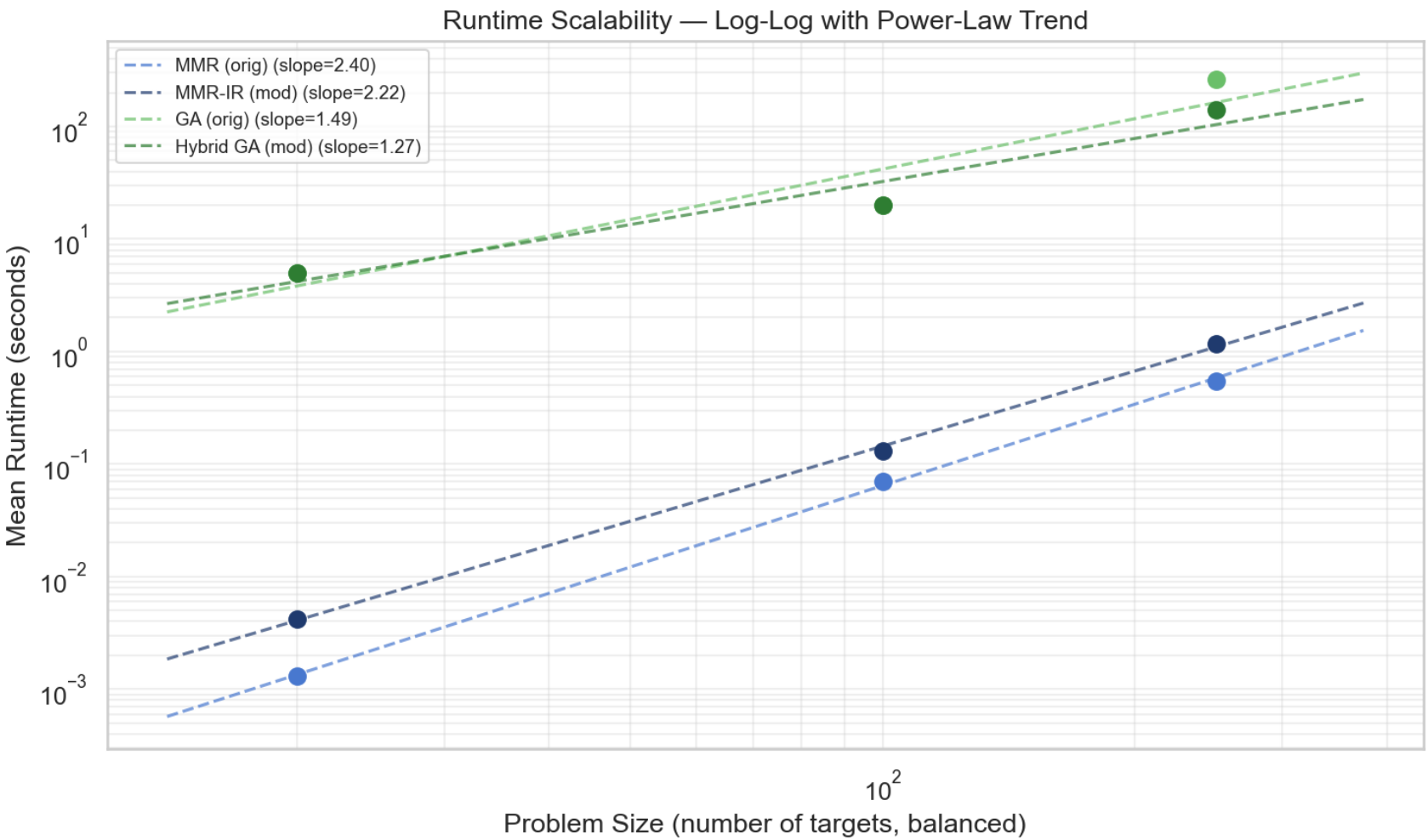
GA (orig) falls dramatically behind on medium and large tiers due to random initialization + no elitism.

Results: Improvement of Modifications



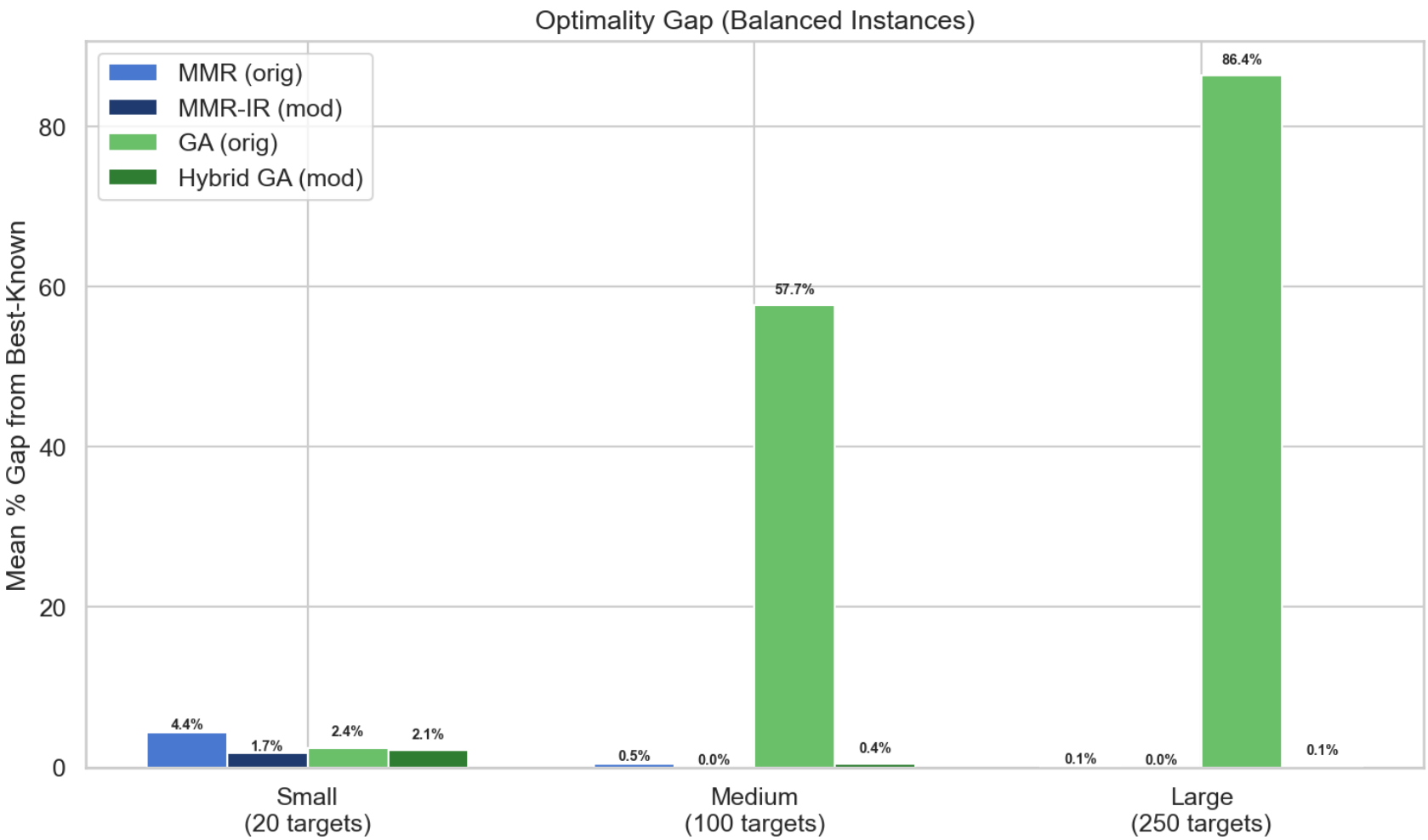
Percentage improvement of modified algorithms over their original counterparts. GA improvement grows with problem size.

Results: Runtime Scalability



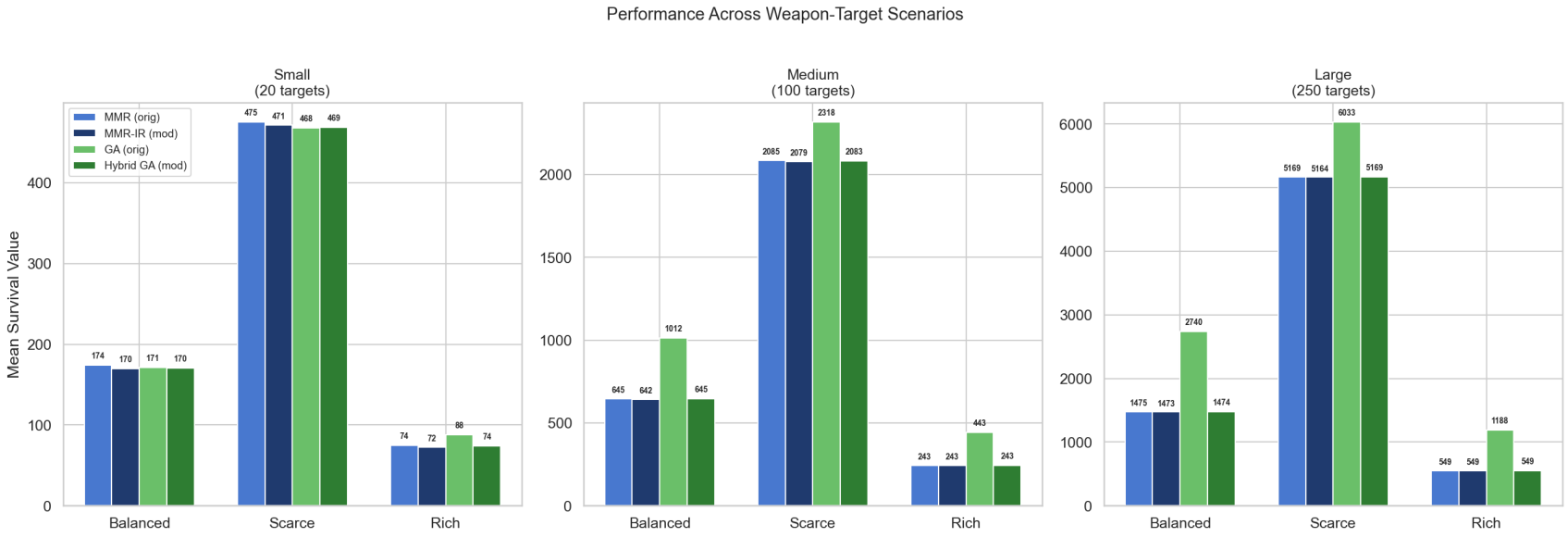
Log-log scale. MMR variants are orders of magnitude faster. GA variants use their full time budget.

Results: Optimality Gap



Gap from best-known solution. GA (orig) has the largest gap, especially on large instances.

Results: Scenario Comparison

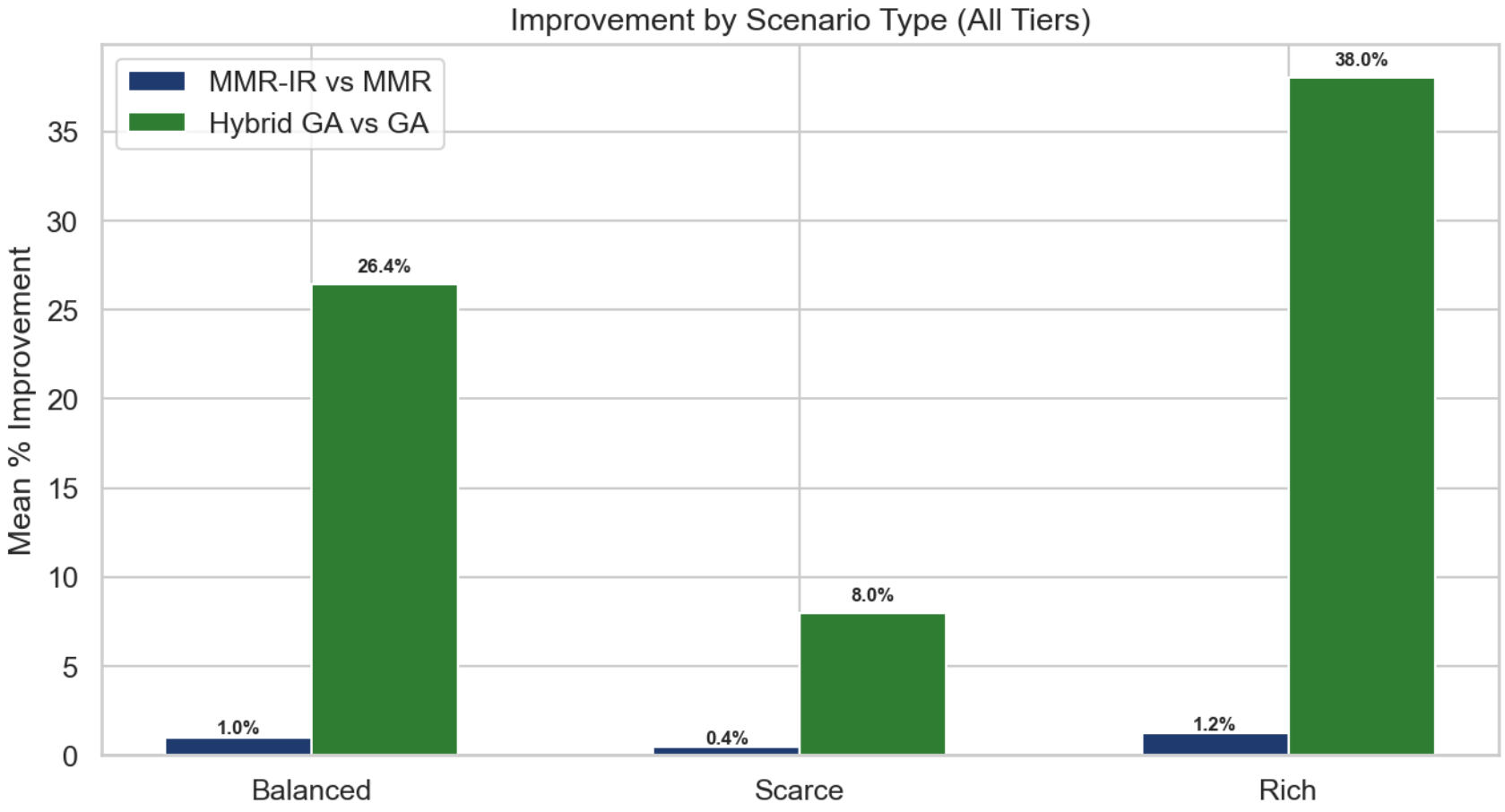


Performance across balanced, scarce, and rich scenarios. Scarce has highest objective (hardest), rich has lowest (easiest).

Results: Value Distributions

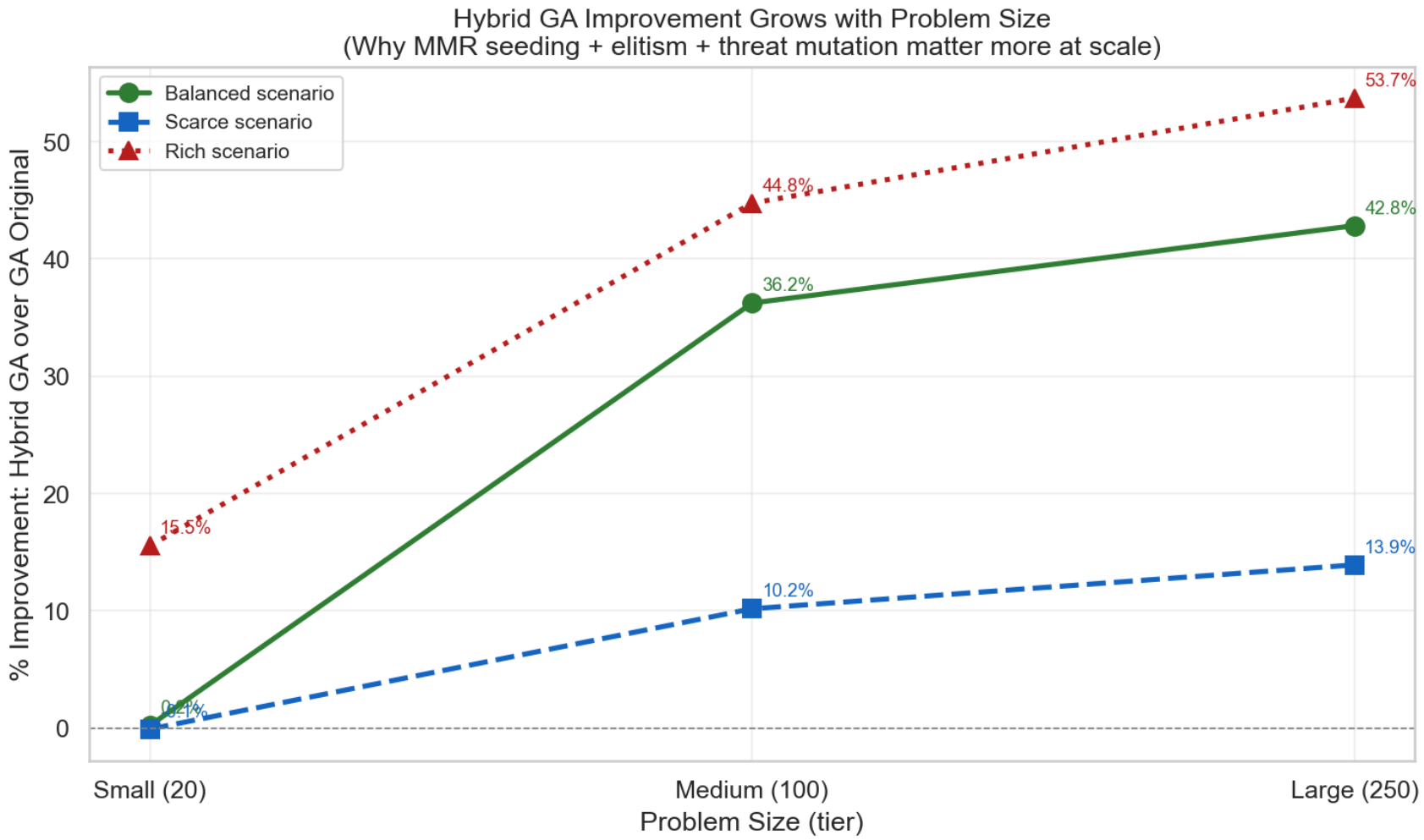
Violin plots showing the distribution of objective values for original vs. modified algorithms.

Results: Improvement by Scenario



Improvement percentage broken down by scenario type (balanced, scarce, rich).

Results: GA Improvement vs. Problem Size



Key finding: Hybrid GA improvement over standard GA grows dramatically with problem size – from ∼1% (small) to ∼54% (large).

Key Findings

MMR-IR vs MMR (Original)

- **Consistent improvement:** 0.2–4.8% across all tiers and scenarios
- 1-opt and 2-opt local search reliably escape greedy suboptimality
- Negligible additional runtime (local search is $O(1)$ per trial)

Hybrid GA vs GA (Original) – The Main Story

- **Small:** 1–16% improvement (modest)
- **Medium:** 10–45% improvement (significant)
- **Large: 36–54% improvement** (dramatic)
- Improvement **grows with problem size** – exactly where it matters most

Statistical Significance

Wilcoxon signed-rank test ($\alpha = 0.05$): All pairwise comparisons between original and modified algorithms are **statistically significant** across all tiers.

Why GA (Original) Fails at Scale

Root Cause Analysis

Three compounding factors explain why standard GA underperforms on medium/large instances:

1. **Large population, limited time:**
   - Population size $= \max(W, T)$. For large_rich: $\max(375, 250) = 375$ individuals
   - With 60s time budget: only $\sim$160 generations completed
   - 160 generations are insufficient to evolve 375-dimensional random solutions
2. **Random initialization:**
   - All individuals start as random assignments – far from any good solution
   - In 375-dimensional space, random solutions have extremely high objective values
3. **No elitism:**
   - The best solution found in early generations can be **lost**
   - Replaced by offspring that may be worse

Our Hybrid GA Fixes All Three

MMR seed (warm start) + Elitism (preservation) + Threat-proportional mutation (guided search)

Results Summary Table

Category	MMR (orig)	MMR-IR	GA (orig)	Hybrid GA
small_balanced	174	170	171	171
small_scarce	475	471	468	469
small_rich	74.5	72.2	87.8	73.8
medium_balanced	645	642	1012	645
medium_scarce	2085	2079	2318	2083
medium_rich	244	243	443	243
large_balanced	1475	1473	2740	1474
large_scarce	5169	5164	6033	5169
large_rich	549	549	1188	549

Mean objective values (lower = better). **Bold observation:** GA (orig) diverges dramatically on medium/large tiers. Hybrid GA brings it back to MMR-level performance. MMR-IR consistently improves over MMR by small but statistically significant margins.

Reproducibility: GitHub Repository

Single Master Script

All experiments are reproducible with a single command:

```
python run_all.py
```

Generates dataset → runs all 4 algorithms on 180 instances → produces 9 plots.
Runtime: ∼3.5 hours total.

Repository Structure

```
CSE_462-Algorithm-Engineering-Sessional-/
  codes/
    run_all.py           # Master script
    dataset_generator.py # 180 instances
    experiment_runner.py # Run all algorithms
    analysis.py          # 9 plots + stats
    mmr_original.py      # MMR (Hasan & Barua)
    mmr_modified.py      # MMR-IR (ours)
    ga_original.py       # GA (Hasan & Barua)
    ga_modified.py       # Hybrid GA (ours)
  datasets/              # JSON instances
  results/figures/       # 9 PNG plots
```

GitHub Repository:

https://github.com/Arimotokika/CSE_462-Algorithm-Engineering-Sessional-

Quick Test (5 instances, ∼1 minute)

```
python experiment_runner.py -categories small_balanced -instances 1-5
python analysis.py
```

References

Hasan, M. & Barua, A. (2020). *Weapon Target Assignment*. IntechOpen. DOI: 10.5772/intechopen.93665

Ahuja, R. K., et al. (2007). *Exact and heuristic algorithms for the weapon-target assignment problem*. Operations Research.

Lee, Z. J., et al. (2003). *Efficiently solving general weapon-target assignment problem by genetic algorithms with greedy eugenics*. IEEE Trans. SMC.

Manne, A. S. (1958). *A target-assignment problem*. Operations Research.

Bertsimas, D., et al. (2025). *Solving large-scale WTA using branch-price-and-cut*. Naval Research Logistics.

Andersen, A. C., et al. (2022). *The weapon-target assignment problem: Exact and approximate algorithms*. Ann. Oper. Res.

Kwon, O., et al. (1999). *A Lagrangian relaxation approach to targeting*. Naval Research Logistics.

Kline, A. G., et al. (2020). *Heuristic and metaheuristic for static weapon-target assignment*. J. Global Optim.

Lee, Z. J., et al. (2002). *Immunity-based ant colony optimization for weapon-target assignment*. Applied Soft Computing.

Lloyd, S. P., et al. (1986). *Weapons allocation is NP-complete*. IEEE.

Thank You!

Questions?